

Lieferpaket für Demos & Games — Amiga-Core (MEGA65), v2

Juni 2026

Lieferpaket für Demos & Games — Anleitung für deft (v2)

1. Worum es geht

Der MEGA65-Amiga-Core bekommt einen **Loader**: Er lädt ein Speicherabbild (“RamDump”) von der SD-Karte ins Chip- und Slow-RAM, stellt den mitgelieferten **CPU-Zustand** her (alle Register) und startet an der gelieferten PC-Adresse. Es gibt dabei **kein Kickstart** zur Laufzeit (im ROM-Bereich liegt unser Mini-Launcher), kein exec, kein dos, kein trackdisk, **kein Floppy-Laufwerk**.

Du lieferst Rohmaterial (Speicherinhalte + Registerstand als formlose Notiz). Das Paketieren ins eigentliche Dateiformat übernehmen wir — du brauchst kein Tool von uns.

2. Die Ziel-Maschine

Eigenschaft	Wert
CPU	68000 (zyklusgenau, fx68k)
Chipsatz	OCS (A500), PAL — keine ECS-/AGA-Register
Chip RAM	512 KB, \$000000–\$07FFFF
Slow RAM (“Trapdoor”, damals oft “fast” genannt)	512 KB, C’000000–C7FFFF
Fast RAM	keins
Kickstart	zur Laufzeit NICHT vorhanden (Launcher-ROM im \$F8xxxx-Bereich)
Floppy	nicht vorhanden (kommt mit späterem Milestone)
Eingaben	Tastatur, Joystick funktionieren; Maus folgt später

3. Was wir beim Start garantieren

Was	Zustand beim Einsprung
CPU-Register	D0–D7, A0–A6, USP, SSP, SR (inkl. CCR-Flags) und PC werden exakt auf deine gelieferten Werte gesetzt (per MOVEM + RTE). Ohne Angabe: Register 0, SR = \$2700, SSP = \$00080000
Chip RAM	\$000000–\$07FFFF = exakt dein Abbild; nicht gelieferte Bereiche sind 0
Slow RAM	C’000000–C7FFFF = exakt dein Abbild; nicht gelieferte Bereiche sind 0
Custom-Chips	Reset-Zustand: DMACON = 0, INTENA = 0, INTREQ = 0, alle DMA aus, kein Copper aktiv. Farb- und sonstige Hardware-Register werden NICHT restauriert — beim Wiederanlauf kann es kurz flackern, bis dein Code sie neu setzt
CIAs	Reset-Zustand
OVL	aus — Chip-RAM ab \$000000 sichtbar, inklusive der Vektortabelle aus deinem Abbild

Was	Zustand beim Einsprung
ROM-Bereich	unser Launcher — ein Sprung dorthin ist ein Absturz

4. Die Bedingungen an den Freeze-/Startpunkt

Der gelieferte Zustand muss ab dem gelieferten PC **allein aus RAM-Inhalt und CPU-Registern weiterlaufen können**. Konkret heißt das:

1. Ab diesem Punkt **keine Kickstart-/ROM-Aufrufe** mehr (auch kein Zugriff auf ExecBase über Adresse \$4).
2. Ab diesem Punkt **kein Disk-Zugriff** mehr — alles ist im RAM (Single-Load). Multi-Part-Titel, die nachladen, gehen erst mit dem Floppy-Milestone.
3. Kein laufender Hardware-Vorgang, auf den der Code wartet (kein Disk-DMA in flight; laufende Copper/Audio-Aktivität ist egal, sie ist nach dem Neustart einfach weg, bis dein Code sie neu aufsetzt).
4. Nur **OCS-Register, PAL**.

Innerhalb dieser Grenzen ist der Zeitpunkt frei — mitten im Entpacken ist okay (außer eventuell kurzem Farbgeflimmer wird dabei ja nichts vorausgesetzt). Je “ruhiger” der Moment, desto unsichtbarer der Übergang.

5. Was du lieferst

Weg 1 — direkt aus dem Build, ohne Dump (empfohlen für den ersten Versuch, schließt den Emulator als Fehlerquelle aus):

1. Die fertigen (entpackten) Binär-Segmente als Dateien
2. Pro Segment die Ladeadresse (“part1.bin nach \$400, part2.bin nach \$20000, musik.bin nach \$C00000, ...”)
3. Kurze Notiz: **Start-PC, SP** (Wert oder “setzt mein Code selbst”); weitere Registerwerte nur, falls dein Einsprung welche erwartet

Weg 2 — WinUAE-Freeze (wenn der Zustand aus dem laufenden System kommen soll):

1. chip.bin — exakt **524.288 Bytes** = \$000000–\$07FFFF
2. slow.bin — exakt **524.288 Bytes** = C00000–C7FFFF (falls Slow RAM genutzt; sonst weglassen)
3. **Der komplette Registerstand zum Freeze-Zeitpunkt:** D0–D7, A0–A7 (USP und SSP), SR/Flags, PC — einfach die r-Ausgabe des Debuggers als Text kopieren oder Screenshot, wir übernehmen das 1:1

Übergabe formlos: ZIP per Discord/Mail. Größen unkritisch (max. ~1 MB).

Checkliste vor dem Abschicken:

- ☐ Ab dem gelieferten PC: kein ROM-/Kickstart-Zugriff, kein Disk-Zugriff (alles im RAM)
- ☐ Nur OCS-Register, PAL
- ☐ Weg 1: Segmente + Ladeadressen + Start-PC vollständig
- ☐ Weg 2: chip.bin exakt 524.288 Bytes (slow.bin ebenso, falls dabei) + kompletter Registerstand (D0–D7, A0–A7, SR, PC)

6. Schritt für Schritt: aus dem Build (Weg 1)

Wenn dein Build-System die finale Speicherbelegung kennt (Packer-/ Linker-Map), exportiere die fertigen Segmente als Dateien und schreib Ladeadressen und Start-PC in die Notiz. Wir setzen das Abbild daraus zusammen. Kein Emulator, kein Debugger, kleinste Dateien — und beim ersten Test wissen wir: Wenn etwas klemmt, liegt es nicht am Dump.

7. Schritt für Schritt: der WinUAE-Freeze (Weg 2)

1. **WinUAE konfigurieren** (entspricht unserem Core): Quickstart A500 mit Kickstart 1.3; CPU 68000 “cycle exact”; Chipset OCS; RAM: Chip 512 KB, Slow 512 KB, Fast 0.

2. **Demo laufen lassen** bis zu einem Moment, der die Bedingungen aus Abschnitt 4 erfüllt (z. B. während des Entpackens, nachdem alles von Disk geladen ist).
3. **Einfrieren:** Shift+F12 öffnet den Debugger — die Emulation steht. Kein Breakpoint nötig. (Falls sich kein Konsolenfenster öffnet, muss in den WinUAE-Einstellungen das Log-/Debugger-Fenster aktiviert werden; Referenz: https://www.amigacoding.com/index.php/WinUAE_debugger)
4. **Registerstand sichern:** r eintippen — Ausgabe kopieren oder Screenshot. Das ist D0–D7, A0–A7, SR/Flags, PC — alles, was wir brauchen.
5. **Speicher sichern** (Zahlen sind hex):


```
S chip.bin 0 80000
S slow.bin c00000 80000
```
6. Fertig. Für einen punktgenauen Freeze (statt “irgendwo beim Entpacken”) gibt es zwei Präzisions-Optionen: eine Endlosschleife (bra.s *) an der gewünschten Stelle einbauen und dort einfrieren — oder im Debugger f <adresse> als Breakpoint setzen (h zeigt die Hilfe, g lässt weiterlaufen). Beides optional.

8. Alternative: Action Replay III in WinUAE

WinUAE kann die Action-Replay-Cartridge emulieren (ROM-Image nötig, in den Einstellungen als Cartridge-ROM einbinden). Dann funktioniert der Freeze-Knopf wie damals, und im AR-Monitor speichert SM <name> <start> <ende> rohe Speicherblöcke — die gleichen Rohdaten wie in Weg 2, nur mit vertrauter Umgebung statt UAE-Debugger. Wichtig: Das **AR-eigene Speicherformat (SA/Save All) ist komprimiert und undokumentiert** — wir verwenden es nicht als Dateiformat; liefere rohe SM-Blöcke plus Registerstand (zeigt der AR-Monitor mit R).

9. Was danach passiert

Wir packen deine Lieferung in unser RamDump-Format (eine Datei pro Titel), der Core-Loader lädt sie über das OSM-Menü von der SD-Karte, und beim ersten Test iterieren wir gemeinsam — **der erste Wurf muss nicht perfekt sein**. Danach lassen sich mit derselben Methode weitere Single-Load-Titel aufbereiten. Multi-Load-Titel folgen mit dem Floppy-Support.

10. FAQ

- **Was, wenn beim Wiederanlauf doch etwas fehlt?** Dann sehen wir das beim ersten Test (schwarzes Bild / Absturz / Flackern) und iterieren. Die Lieferung kostet einmal 15 Minuten, kaputtgehen kann nichts.
- **Warum kann es kurz flackern?** Farb- und andere Hardware-Register sind nicht Teil des Abbilds und werden nicht restauriert; sie stehen beim Start auf Reset-Werten, bis dein Code sie (wieder) setzt.
- **Interrupt-Vektoren?** Liegen bei \$0–\$3FF im Chip-RAM und sind Teil deines Abbilds. Achtung: Wenn zum Freeze-Zeitpunkt noch Kickstart-Vektoren installiert sind UND dein SR Interrupts zulässt, würde der erste Interrupt ins nicht vorhandene ROM springen. Beim Entpacker-Freeze eines Trackmos (Interrupts aus oder eigene Vektoren) ist das kein Thema.
- **Tastatur/Joystick?** Funktioniert — CIA-A-Scancodes und Joystick-Ports sind verdrahtet. Maus kommt später; falls die Demo Maus braucht, kurz erwähnen.
- **Warum kein Kickstart zur Laufzeit?** Der Loader ersetzt das Kick-ROM durch einen Mini-Launcher, der nur OVL ausschaltet, deine Register herstellt und startet. Deshalb die Bedingung “keine ROM-Aufrufe ab dem gelieferten PC”.